Lecture 12
Synthesis + Verification

Armando Solar-Lezama

Recap

Given a definition of VC

- $VC(skip, Q) = Q$
- $VC(x = e, Q) = Q[e/x]$
- $VC(C1; C2, Q) = VC(C1, VC(C2, Q))$
- $VC(if\ B\ then\ C1\ else\ C2, Q) = \big(B\ and\ VC(C1, Q)\big) or\ (\neg B\ and\ VC(C2, Q))$
- $VC(while_I\ e\ do\ c, Q) =$
  $$I \wedge \ \forall x_1, \ldots x_n\ I \Rightarrow (e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow Q)$$
  - Where x_i are variables modified in c.
- $VC(assert\ B, \ Q) = \{Q \wedge B\}$
- $VC(assume\ B, Q) = \{B \Rightarrow Q\}$

$$\big(A \Rightarrow VC(P, B)\big) \Rightarrow \{A\}P\{B\}$$

- $\big(A \Rightarrow VC(P, B)\big)$ is equivalent to $VC(assume\ A; P; assert\ B, \ true)$

Synthesizing invariants

Given a program with pre/post conditions, the only obstacle to making verification automatic is finding the invariants

- (and checking the resulting formula if it happens to be in a complicated logic)

Example

```
{y=y_0, k=k_0, t=y_0-k_0}
while(t>0){
        y = y-1;
        t = t-1;
}
Assert y <= k_0;
```

VC as a sketch harness

$$\forall y, y_0, k, k_0, t, y', t', y'', (y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) \Rightarrow \quad inv(y, y_0, k, k_0, t)$$

$$inv(y', y_0, k, k_0, t') \wedge (t' > 0) \Rightarrow inv(y' - 1, y_0, k, k_0, t' - 1)$$

$$inv(y'', y_0, k, k_0, t'') \wedge \neg(t'' > 0) \Rightarrow y'' \leq k_0,$$

```
harness void main(int y, int y_0, int k, int k_0, int t,
                  int yp, int tp, int ypp, int tpp ){
   if(y==y_0 && k==k_0 && t==y_0-k_0){
      assert inv(y, y_0, k, k_0, t);
      if(inv(yp, y_0, k, k_0, tp) && tp>0){
         assert inv(yp-1, y_0, k, k_0, tp-1);
      }
      if(inv(ypp, y_0, k, k_0, tpp) && tpp<=0){
         assert ypp <= k_0;
      }

} }
```

VC as a sketch invariant

```
include "generators.skh";
pragma options "--bnd-inline-amnt 3";

bit inv(int y, int y_0, int k, int k_0, int t){
      return exprBool({y, y_0, k, k_0, t}, {PLUS});

}
```

Summary

If we find a solution that works for all inputs, we know the program verifies

Sketch has no loops so bounded execution is not a problem

Sketch is only considering small inputs

- To get full guarantees we need to check that the resulting invariant works for *all* inputs
- We can check that with an SMT solver

Verified synthesis

What if we want to synthesize the function body too?

Candidate solution:

- Inline all generators
- Pretend that ?? Is just another expression
- Generate VC from that program.
- Solve for all unknowns
- Plug them back into the original program

This almost works

Verified synthesis

{P}
While(true){ }
{Q}

This is valid for any P and Q!

From partial to total correctness

Total correctness judgment

- $\vdash [A]\ c\ [B]$
- Just like before, but must also prove termination

$$\frac{\vdash [A \wedge b] c_1\ [B] \quad \vdash [A \wedge not\ b] c_2\ [B]}{\vdash [A] if\ b\ then\ c_1 else\ c_2\ [B]} \qquad \frac{}{\vdash [A[x \to e]] x := e\ [A]}$$

$$\frac{\vdash [A] c_1\ [C] \quad \vdash [C] c_2\ [B]}{\vdash [A] c_1; c_2\ [B]}$$

What about loops

Ranking function

Function F of the state that

- a) Maps state to an integer (in general, a well-ordered set)
- b) Decreases with every iteration of the loop
- c) Is guaranteed to stay greater than zero
- Also called variant function

$$\frac{\vdash [A \wedge b \wedge F = z]c\ [A \wedge F < z] \quad \vdash A \wedge b \Rightarrow F \geq 0}{\vdash [A] while\ b\ do\ c\ [A \wedge not\ b]}$$

If a ranking function exists, program is guaranteed to terminate

We can synthesize that too!

From program verification to program synthesis

Srivastava, Gulwani, Foster

Key ideas

Structured program statements to reduce symmetries

Stylized program fragments

Language supports only the following simple grammar

$P \coloneqq \{g \rightarrow s\} |\ P; P\ | while(e) P$

$g \rightarrow s$ corresponds to a guarded assignment of the form:

- $If\,(g)\,\{\,s_1 \ldots s_k = e_1 \ldots e_k\}$

- Parallel assignment avoids the symmetries that arise from having to compute all possible permutations of assignments.

Proof-Theoretic Synthesis

Input: Program Scaffold

- Flowgraph template (a nested loop structure)
- Function spec (pre-/post-conditions)
- Other constraints (temp variables, size limit, etc.)

Output:

- Synthesized code fragments
- Associated loop invariants and ranking functions

Synthesis Condition:

- Generalization of verification conditions
- A valid program exists iff. the SC has a satisfying solution

Example: Square Root

```
IntSqrt(int x) {
  v:=1;i:=1;
  while^{τ,φ}(v ≤ x)
    | v:=v+2i+1;i++;
  return i−1;
}
```

Invariant τ:
$v{=}i^2 \wedge x \geq (i{-}1)^2 \wedge i \geq 1$

Ranking function φ:
$x - (i-1)^2$

Program Scaffold

Pre: $x \geq 1$

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

Post: $(i-1)^2 \leq x < i^2$

(Assume only one branch in each fragment)

Verification Condition

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad\texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

$$
\begin{array}{rcll}
x \geq 1 \wedge g_1 \wedge s_1 & \Rightarrow & \tau' & \wedge \\
\tau \wedge g_0 \wedge g_2 \wedge s_2 & \Rightarrow & \tau' & \wedge \\
\tau \wedge \neg g_0 \wedge g_3 \wedge s_3 & \Rightarrow & (i'-1)^2 \leq x' \wedge x' < i'^2 &
\end{array}
$$

Well-formedness Condition

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

$$g_1, g_2, g_3 = \texttt{true}\ \textit{and}\ \texttt{valid}(s_1) \wedge \texttt{valid}(s_2) \wedge \texttt{valid}(s_3)$$

Ranking Function

```
choose {[]g1 → s1};
while^{τ,φ} (g0) {
    choose {[]g2 → s2};
};
choose {[]g3 → s3}
```

$$r_l = \varphi_l \wedge (\tau \Rightarrow r_l \geq 0) \wedge (\tau \wedge g_0 \wedge g_2 \wedge s_2 \Rightarrow r'_l > \varphi'_l)$$

Synthesis Conditions

$$sc = \texttt{SafetyCond}(exp, \mathcal{F}) \wedge \texttt{WellFormCond}(exp) \wedge \texttt{RankCond}(exp)$$

$$\begin{array}{rcll} x \geq 1 \wedge s_1 & \Rightarrow & \tau' & \wedge \\ \tau \wedge g_0 \wedge s_2 & \Rightarrow & \tau' & \wedge \\ \tau \wedge \neg g_0 \wedge s_3 & \Rightarrow & (i'-1)^2 \leq x' \wedge x' < i'^2 & \wedge \\ \texttt{valid}(s_1) \wedge \texttt{valid}(s_2) \wedge \texttt{valid}(s_3) & & & \wedge \\ r_l = \varphi_l \wedge (\tau \Rightarrow r_l \geq 0) \wedge (\tau \wedge g_0 \wedge s_2 \Rightarrow r_l' > \varphi_l') & & & \end{array}$$

Theorem (soundness and completeness)

The synthesis conditions are satisfiable iff. there exists a program that is valid w.r.t. the scaffold.